

Leveraging Generative AI for Architecture Knowledge Management

Rudra Dhar
SERC, IIIT Hyderabad, India
rudra.dhar@research.iiit.ac.in

Karthik Vaidhyathan
SERC, IIIT Hyderabad, India
karthik.vaidhyathan@iiit.ac.in

Vasudeva Varma
IREL, IIIT Hyderabad, India
vv@iiit.ac.in

Abstract—While documenting Architectural Knowledge (AK) is crucial, it is frequently neglected in many projects, and existing manual tools are underutilized. Although undocumented, Architecture Knowledge (AK) is dispersed across various sources such as source code, documentation, and runtime logs. To address this, automated tools for efficient AK extraction and documentation are essential. Even after generating AK, navigating through vast the Architectural Records can be overwhelming. Building on that, we propose an automated Architectural Knowledge Management (AKM) System using Information Extraction and Generative AI, which generates AK from various source for a given system and answers architectural queries with respect to the given system.

The development of an efficient Architectural Knowledge Management (AKM) system, which is both effective and user-friendly, entails the resolution of numerous challenges. It requires consolidating diverse AK data sources scattered across code, diagrams, repository commits, and online platforms. The integration of Multimodal AI for AK extraction, incorporation of global AK, and leveraging Generative AI for AK documentation further compounds the problem. Moreover, generating contextually appropriate query responses adds another layer of complexity.

To this end, we performed an initial exploratory study on generating Architectural Design Decisions using generative Large Language Models (LLM) in the context of Architecture Decision Records (ADR). Our initial results have been promising indicating the potential impact of GenAI for architectural knowledge management.

Index Terms—Software architecture, AKM, Machine learning, Generative AI, Information Extraction, LLM

I. INTRODUCTION

Architectural Knowledge Management (AKM) involves systematically managing architectural information within projects or organizations, encompassing styles, design patterns, and quality attributes. AKM aims to ensure decision traceability, facilitate knowledge reuse, and prevent knowledge vaporization thus enabling effective decision-making, issue resolution and collaboration.

Capturing and documenting *Architectural Knowledge (AK)* is widely recognized as crucial. Although some AKM system and tools have been created, they are deterministic and manual. Their use is limited, resulting in scarce AK documentation [1]. Rainer et al. [3] highlight that despite significant efforts to advance AKM and capture AK, these endeavors have not fully succeeded, hindering broader adoption. This emphasizes the need for further research in automating AK capture [2].

Advancements in Generative AI and Information Extraction enable leveraging diverse AK data sources like code

and diagrams to generate AK. Combining Large Language Models (LLMs) like GPT with information extraction systems enhances AK generation capabilities. Moreover, manual navigation through generated AK can be laborious, highlighting the need for a AK retrieval approach. Such systems streamline querying and extraction of relevant AK, improving accessibility and usability.

Hence in this paper, we propose an approach that applies the emergent techniques of Generative AI and Information Retrieval for AKM. The proposed approach will aid architects in generating and documenting AK within software projects and reduce manual labor compared to existing AKM systems. This should mitigate knowledge vaporization and ensure decision traceability. Its AK retrieval method further facilitates efficient and effective decision-making.

To this end, we performed an initial exploratory empirical study on generating Architectural Design Decisions using generative LLM in the context of Architecture Decision Records (ADR).¹ We used GPT and T5-based models with 0-shot, few-shot, and fine-tuning approaches. The study concludes that while advanced models like GPT-4 yield relevant Design Decisions, they still lag behind human-level performance. Smaller models like Flan-T5, when locally fine-tuned, also deliver comparable results. Overall, the study suggests that LLMs offer potential for generating Design Decisions but highlights the need for further research to achieve human-level performance and promote widespread adoption, validating our approach to employing generative AI for AKM.

The paper continues as follows: Section II outlines related work on AKM tools and technologies. Section III discusses anticipated challenges while Section IV elaborates on our approach. Section V covers initial validation, and Section VI concludes.

II. RELATED WORK

For several decades, the significance of AKM has been widely recognized, and a huge number of manual and deterministic tools have been developed to assist AKM [1]. Rafael et al. [2] did a review of AKM systems, and found that the cost to capture AK is high for existing AKM systems, which is preventing widespread adoption of these tools.

¹https://github.com/sa4s-serc/ArchAI_ADR

Manoj et al. [5] presented an approach for an AKM framework aimed at supporting software architects in their day-to-day activities and addressing challenges in managing AK for decision making, what-if analysis, and collaboration in large industrial projects. Their approach aims to consolidate project data, define domain-specific rules, and address challenges in inconsistency analysis and artifact tracking.

In contrast to the works mentioned above we propose to pursue AKM with information extraction, retrieval, and Generative AI techniques. As highlighted by Anh et al. [6], there exists a substantial array of research avenues concerning the application of Generative AI in Software Engineering, with ongoing efforts from researchers to explore and address these directions. But to the best of our knowledge, Generative approaches have not been applied to AKM.

III. MOTIVATION AND CHALLENGES

To create an AKM approach that architects in the industry would use effectively [2], we need to consider the following:

AK Data: Software Architecture (SA) consists of Elements (Architecture Diagrams), Form (Style, Patterns), and Rationale (ADR). Aggregating comprehensive datasets with these components poses a significant engineering challenge. Moreover this data appears in diverse forms such as diagrams (like UML diagrams), documentation (like ADRs), and source code. Extracting AK from these varied sources using multi-modal AI models or a combination of uni-modal models poses a significant challenge due to the complexity of distributed AK across multiple modes.

World AK: To capture comprehensive AK in software, our approach needs a generalized AK akin to that of an expert architect. This requires retrieving and integrating global AK into our system, posing challenges in determining what and how to retrieve. Aggregating the AK and representing it within the system context adds further complexity.

Query Answer: Utilizing Generative AI to provide contextually relevant responses to user queries using stored AK is challenging. The system must understand queries, retrieve relevant AK, and generate responses, potentially including multimodal content like Architecture Diagrams, adding complexity.

User Friendly: Our proposal emphasizes user-friendliness to encourage industry adoption. Existing AKM systems often require substantial manual effort and are underutilized by architects. Therefore, our system, especially the Query Response component, must prioritize user-friendliness.

IV. APPROACH

The proposed approach to automate AKM, as depicted in Figure 1, consists of two phases, namely, *Architecture Knowledge Extraction (AKE)* and *Architecture Knowledge Retrieval (AKR)*. The goal of AKE is to extract and store architectural information of different software systems. AKR then builds on the knowledge extracted to retrieve the respective AK corresponding to the query provided by a stakeholder.

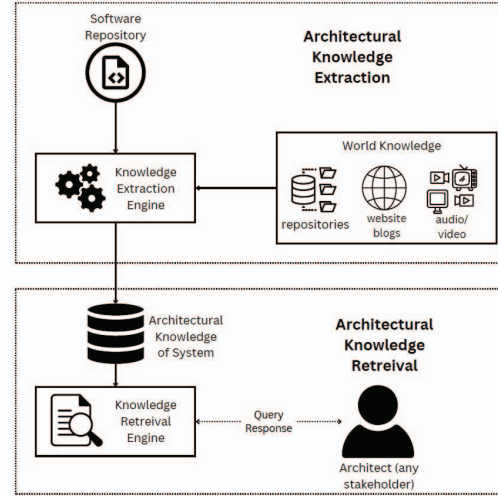


Fig. 1. Proposed Approach

A. Architecture Knowledge Extraction (AKE):

As mentioned above, the goal of AKE is to extract AK of software systems with support from various available sources to generate comprehensive architectural information of the given software as depicted in Figure 2. AKE further consists of four steps.

Extracting AK from system: Our approach involves parsing, analyzing, and extracting latent Architectural Knowledge (AK) from software repositories or specific components like Java files or packages. It will generate AK in various formats such as JSON, XML, UML diagrams, and text documentation like ADRs. Techniques will be developed to convert input formats into desired output formats, leveraging AI methodologies like LLMs, Graph Neural Networks, and parsing Abstract Syntax Trees, alongside deterministic AKM tools depicted at the top of Figure 2.

World AK Retrieval: Our methodology involves gathering supplementary information from diverse sources available in various formats, as illustrated in the 'World AK' container in Figure 2. These sources include additional code repositories, AK repositories like AKB², ADR repositories like Arachene³, scholarly articles, books, blogs [8], articles, images (such as UML diagrams), Grey literature, and transcripts⁴. LLMs themselves can also serve as sources of world AK. In Section V, we discuss our initial study employing LLMs as a source for world AK, particularly in the context of Design Decisions.

AK Fusion: The retrieved world AK may vary in formats and content from the considered system's AK. Thus, our approach, depicted in the 'AK Generator' component of Figure 2, will extract precise information from the global AK and integrate it with the system AK to generate the necessary

²<https://akb.se.jku.at/#/>

³<https://github.com/arachne-framework/architecture>

⁴<https://se-radio.net/>

AK. The validity of the produced AK will be confirmed by an Oracle Verifier, potentially using formal methods for verification.

AK Representation and storage: The extracted knowledge, stored for future reference, requires representation in various formats such as diagrams, textual descriptions, knowledge graphs, and vector databases. Generative AI techniques like Image Generator or LLMs, as depicted in the 'AK Generator' component of Figure 2, will be leveraged for this task. The concepts of *Retrieval Augmented Generation (RAG)* [7] will be used here. Though RAG primarily operates within the domain of text, here we will adopt a Multi-modal RAG approach.

As an illustration of the envisioned system's functionality, consider the following scenario: starting with the Hadoop GitHub repository ⁵, the system would first extract AK from the repository. Subsequently, it would retrieve world AK from a source such as AKB website ⁶. Combining these sources, the system would then generate a comprehensive AK document such as The Hadoop Distributed File System's Architecture Profile ⁷, which consists of information like design decisions, patterns selected, tactics applied, etc.

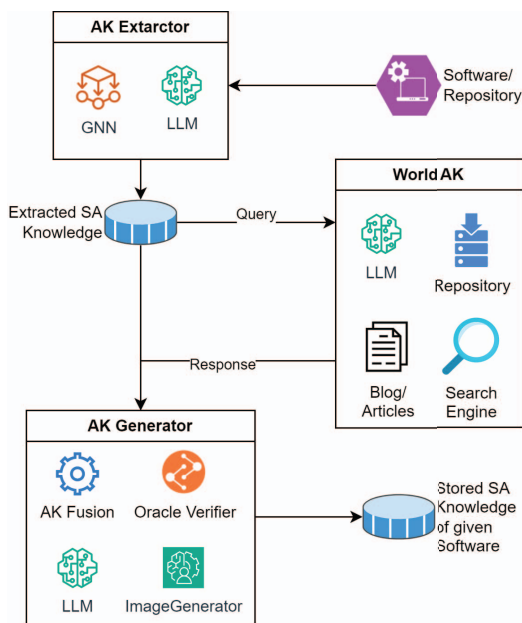


Fig. 2. Approach for AK Extraction

B. Architecture Knowledge Retrieval (AKR):

The AKR operates based on the principles of RAG [7]. Upon receiving a user query, our approach retrieves relevant AK from the stored repository and generates a response based on the query and retrieved AK.

Our approach adopts a role-oriented framework to tailor responses to the specific needs of different users. Whether

the query comes from an architect, developer, fresher, tester, project manager, or generic user, our system delivers responses customized to their roles and responsibilities within the software development lifecycle. This ensures that the information provided is accurate, comprehensive, and actionable, catering to the specific needs of each user role.

User queries in our system can be open-ended or template-based. Template queries address specific information needs, leading to more precise responses. Key templates include the *System Architecture*, for an overview of design; *Decisions/Context*, detailing the rationale behind architectural choices; *Quality*, assessing quality attributes like performance and reliability; *Similar Architecture*, for retrieving existing similar architectures; *Architecture Modification*, suggesting architectural changes for new features; and *Candidate Architecture*, generating candidate architecture for building new systems similar to the existing system.

The approach will result in a system that will feature an user interface designed for generating and documenting AK for software projects. Moreover, it will encompass a retrieval interface enabling architects to access past documented AK of the considered project.

V. INITIAL VALIDATION

Context: We need to decide on whether to use Python as a programming language for our project. Our project involves data analysis, machine learning, and web development.
Decision: We have decided to use Python as our primary programming language for our project.

Fig. 3. Sample ADR with Context Decision pair

Architecture is a set of key design Decisions [9] and one of the important part of AKM is capturing Architectural Design Decisions, and this is typically done using lightweight documents called Architecture Decision Records (ADR). These records encapsulate the contextual background, the decision itself, and other pertinent aspects related to the design decision-making process. Figure 3 notes the context and decision of a sample ADR. Despite their benefits, ADR adoption in software development has been slow due to challenges like time constraints and inconsistent uptake. Hence we performed an exploratory study to Generate the Decision of an ADR given its Context using LLMs [4].

We employed web scraping to collect pairs of context and decisions, essential components of ADRs, from diverse ADR repositories. The extracted context from the ADRs serves as the AK for a specific system, depicted as Extracted SA Knowledge in Figure 2. We then input this context into an LLM to generate corresponding Design Decisions. The LLM serves as a source of world AK. Consequently, we have both the context and the generated decision, which can be stored as AK for the given system, as shown at the bottom of Figure 2.

⁵<https://github.com/apache/hadoop>

⁶<https://akb.se.jku.at/#/>

⁷<https://akb.se.jku.at/#/archprofiles/775>

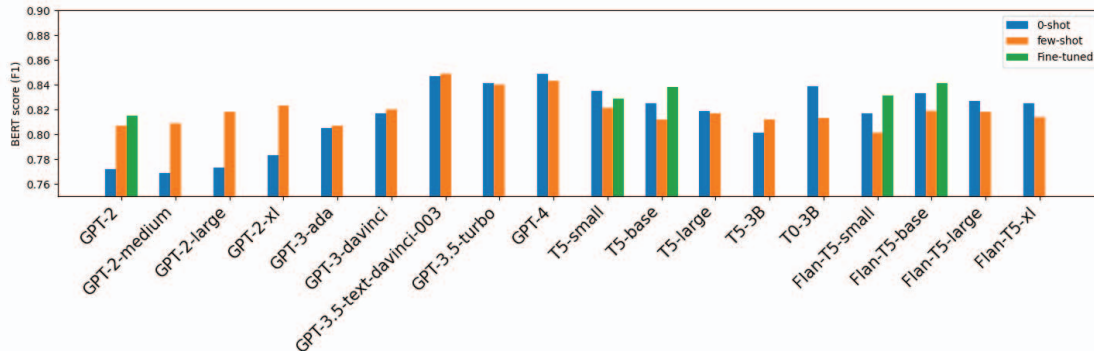


Fig. 4. Result (Bert Score F1)

We utilize GPT and T5-based models with 0-shot, few-shot, and fine-tuning methodologies to generate ADR Decisions based on their respective Contexts. Our evaluation framework incorporates established NLP metrics such as ROUGE-1, BLEU, METEOR, and BERT Score. Figure 4 illustrates BERT Score (F1) for all models across all approaches. Our findings show that state-of-the-art models like GPT-4 can generate relevant and accurate Design Decisions in a 0-shot setting but fall short of human-level performance. Economically viable alternatives like GPT-3.5 achieve similar outcomes in a few-shot setting. Smaller models like Flan-T5, trainable on a single GPU and deployable locally, also exhibit promising results post fine-tuning. In summary, this exploratory study indicates the potential of LLMs to generate Design Decisions, albeit requiring further research to reach human-level proficiency and establish standardized adoption.

VI. CONCLUSION

In this work we proposed an approach to automate AKM. The proposed AKM approach, Leveraging Information Extraction, Retrieval, and Generative AI techniques, aims to streamline AK generation, documentation, and democratization. Our approach has two phases. Our process involves initially extracting and documenting AK from designated repositories. Subsequently, we plan an AK retrieval phase allowing users to access relevant AK based on their queries.

We anticipate that our proposed approach will lead to a highly automated, efficient and user-friendly AKM system, thereby simplifying AKM processes, popularizing AK documentation, preventing knowledge vaporization and ultimately enhancing decision-making, AK accessibility, and team collaboration.

As part of initial validation, we explored if LLMs can be used to generate Design Decisions in the context of ADRs. Our findings revealed that while LLMs can generate Design Decisions, they do not yet match human proficiency.

REFERENCES

- [1] Tang, Antony & Avgeriou, Paris & Jansen, Anton & Capilla, Rafael & Ali Babar, Muhammad. (2013). A comparative study of architecture

- knowledge management tools. *Journal of Systems and Software*. 352-370. 10.1016/j.jss.2009.08.032.
- [2] Capilla, Rafael & Jansen, Anton & Tang, Antony & Avgeriou, Paris & Ali Babar, Muhammad. (2016). 10 years of Software Architecture Knowledge Management: Practice and Future. *Journal of Systems and Software*. 10.1016/j.jss.2015.08.054
- [3] Weinreich, Rainer & Groher, Iris. (2016). Software architecture knowledge management approaches and their support for knowledge management activities: A systematic literature review. *Information and Software Technology*. 80. 10.1016/j.infsof.2016.09.007.
- [4] Rudra Dhar, Karthik Vaidhyathan, Vasudeva Varma. Can LLMs Generate Architectural Design Decisions: An Exploratory Empirical Study, *ICSA 2024*. <https://arxiv.org/abs/2403.01709>
- [5] Manoj Bhat, Klym Shumaiev, Andreas Biesdorf, Uwe Hohenstein, Michael Hassel, and Florian Matthes. 2016. Meta-model based framework for architectural knowledge management. In *Proceedings of the 10th European Conference on Software Architecture Workshops (EC-SAW '16)*, Association for Computing Machinery, Copenhagen, Denmark. DOI:<https://doi.org/10.1145/2993412.3004848>
- [6] Nguyen-Duc, Anh, Beatriz Cabrero Daniel, Adam Przybylek, Chetan Arora, Dron Khanna, Tomas Herda, Usman Rafiq, Jorge Melegati, Eduardo Guerra, Kai-Kristian Kemell, Mika Saari, Zheyang Zhang, Huy Le, Tho Quan and Pekka Abrahamsson. "Generative Artificial Intelligence for Software Engineering - A Research Agenda." *ArXiv abs/2310.18648* (2023): n. pag.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS'20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 9459–9474.
- [8] Soliman, Mohamed & Gericke, Kirsten & Avgeriou, Paris. (2023). Where and What do Software Architects blog? An Exploratory Study on Architectural Knowledge in Blogs, and their Relevance to Design Steps.
- [9] A. Jansen and J. Bosch, "Software Architecture as a Set of Architectural Design Decisions," 5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05), Pittsburgh, PA, USA, 2005, pp. 109-120, doi: 10.1109/WICSA.2005.61. keywords: Software architecture;Computer architecture;Costs;Software systems;Application software;Computer industry;Electrical equipment industry;Connectors;Stress control;Control systems,
- [10] Soliman, Mohamed & Galster, Matthias & Avgeriou, Paris. (2021). An Exploratory Study on Architectural Knowledge in Issue Tracking Systems.
- [11] Soliman, Mohamed & Wiese, Marion & Li, Yikun & Riebisch, Matthias & Avgeriou, Paris. (2021). Exploring Web Search Engines to Find Architectural Knowledge.